

Bài 5.tiếp chương 3

3.9. Cây nhị phân tìm kiếm tự cân bằng AVL

3.9.1.Khái niệm cây AVL

Cây AVL (viết tắt của tên các nhà phát minh **A**delson, **V**elski và **L**andis) là *cây tìm kiếm nhị phân BST có độ cân bằng cao*. Cây AVL kiểm tra độ cao (bậc) của các cây con bên trái và cây con bên phải và bảo đảm rằng hiệu số giữa chúng là không lớn hơn **1**. **Hiệu số** này được gọi là Hệ số cân bằng.

Hệ số cân bằng = Chiều cao của cây con bên trái - Chiều cao của cây con bên phải

(Hoặc Hệ số cân bằng = Chiều cao của cây con bên phải - Chiều cao của cây con bên trái)

Khi cây cân bằng, hệ số này = 0 hoặc ± 1 . Tổng quát : cây cân bằng khi $hs_{cb} \leq \pm 1$

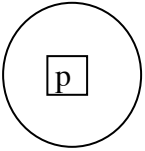
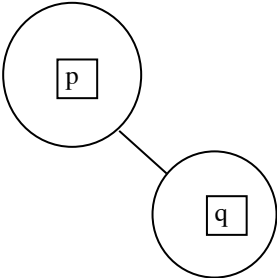
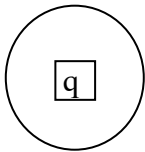
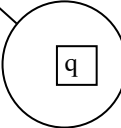
Có thể định nghĩa cây AVL theo cách khác :Cây AVL là cây tìm kiếm nhị phân tự cân bằng, trong đó, mỗi nút duy trì thông tin bổ sung được gọi là hệ số cân bằng có giá trị là -1, 0 hoặc +1.

Ưu điểm của cây AVL là quá trình tìm kiếm nút nhanh, trường hợp xấu nhất độ phức tạp cũng chỉ bằng $O(\log N)$. Nhược điểm của cây AVL là khi thêm hoặc xóa nút thì cần thêm việc kiểm tra độ cân bằng, nếu mất cân bằng thì phải xoay các nút đưa cây về trạng thái cân bằng.

Cây AVL là cây nhị phân tìm kiếm BST nên việc đưa thêm nút vào cũng giống như thêm nút vào cây BST.

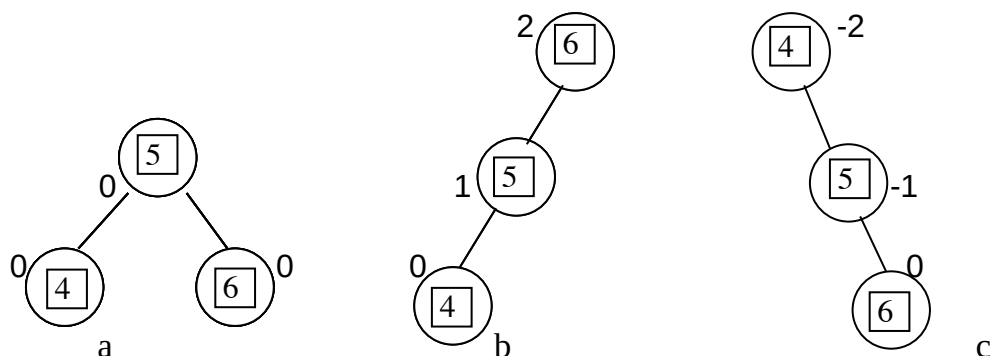
Chú ý rằng nút mới thêm vào bao giờ cũng là nút lá.

Nếu dữ liệu nút mới < nút hiện thời thì nút mới là nút lá trái, ngược lại là nút lá phải.

	q -> dữ liệu < p -> dữ liệu	q -> dữ liệu > p -> dữ liệu
Nút hiện thời p		
Nút mới q		

Nếu nút mới đưa vào cây BST làm cho hiệu số giữa độ cao của các cây con bên trái và cây con bên phải của nút cha lớn hơn 1 thì sử dụng các phép quay để cây được cân bằng.

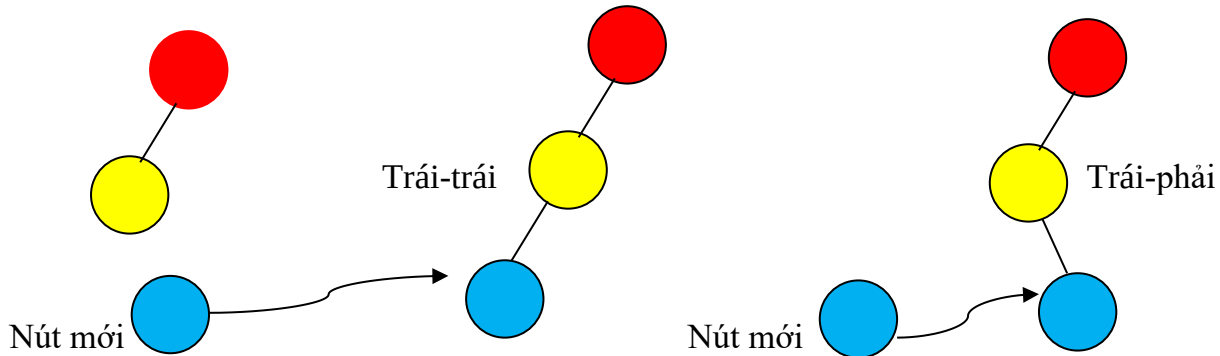
Trong 3 cây ở hình 1 : a) là cân bằng, b) và c) là không cân bằng.



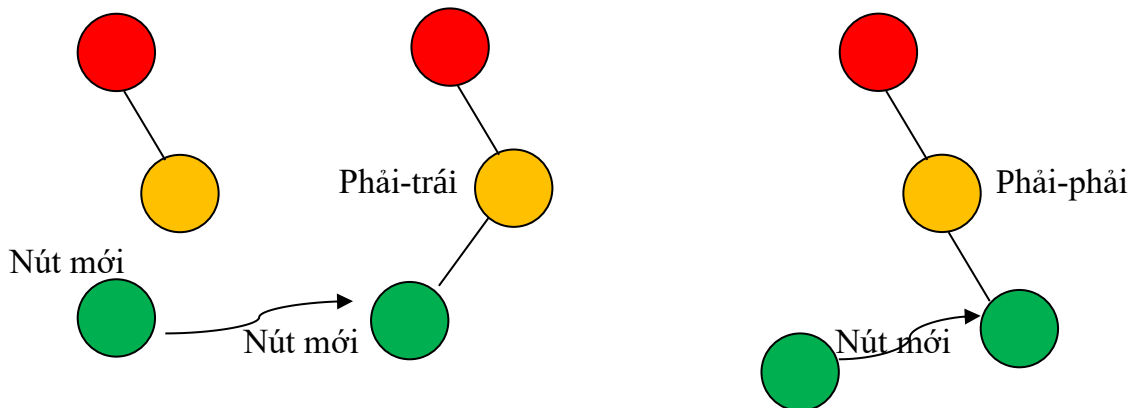
Hình 1.

Trong cây 1.b, cây con bên trái của [6] có độ cao là 2 và cây con bên phải của nó có độ cao là 0, do đó hiệu số là 2. Trong cây 1.c, cây con bên phải của [4] có độ cao là 2 và cây con bên trái của nó có độ cao là 0, do đó hiệu số là -2. Ký hiệu độ cao của nút bất kỳ là height.

Bây giờ ta xét 4 trường hợp khi thêm nút mới làm cho mất cân bằng trong cây AVL. Thêm vào bên trái của cây con trái (trái-trái) hoặc thêm vào bên phải của cây con trái(phải-trái)



Thêm vào bên trái của cây con phải(trái-phải) hoặc vào bên phải của cây con phải(phải-phải)



Tổng hợp lại có 4 trường hợp khi thêm nút mới phát sinh mất cân bằng :
Trái-trái/Phải-trái /Trái-phải /Phải-phải

3.9.2.Các phép quay cây AVL

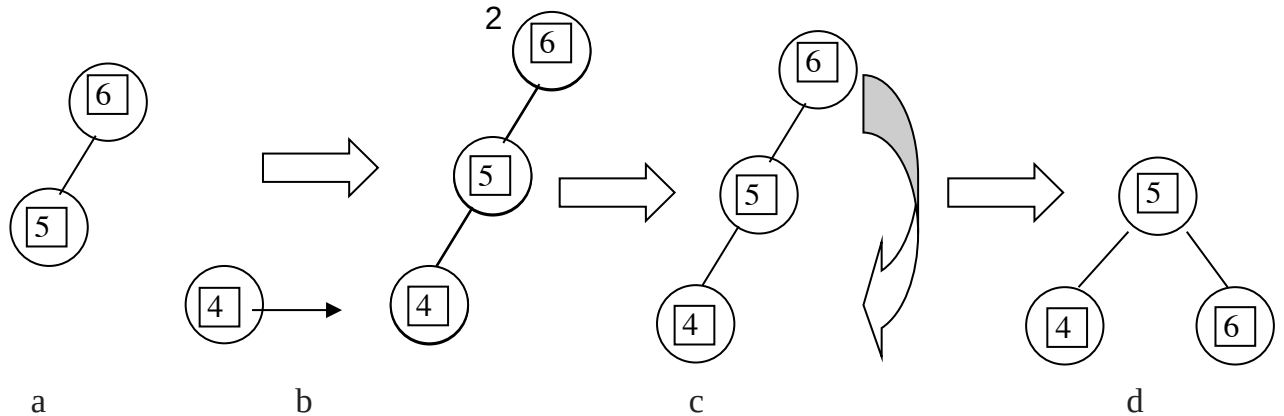
Khi thêm nút vào cây tìm kiếm nhị phân, để làm cho cây tự cân bằng, có thể thực hiện 4 loại phép quay sau:

- Quay phải đơn
- Quay trái đơn
- Quay kép trái-phải
- Quay kép phải-trái

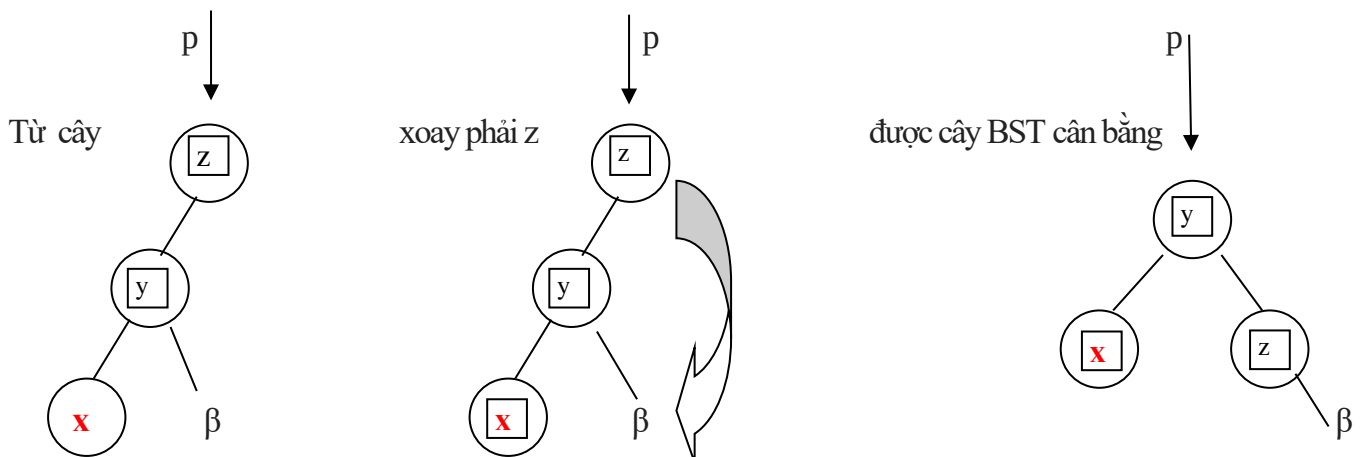
3.9.2.1. Quay phải cây AVL

Để cân bằng cây BST khi chèn thêm một nút mới vào cây con bên trái của cây con bên trái của nút đang xét làm cho nó mất cân bằng ($\text{abs}(\text{hệ số cân bằng của nút}) > 1$), ta dùng phép quay phải.

Trong hình 2 dưới đây : ban đầu ta có cây trái 2 nút 6,5 (h2.a), sau đó thêm nút 4 vào cây con bên trái(h2.b) thành cây không cân bằng. Thực hiện phép quay phải nút 6 (h2.c) thành cây cân bằng (h2.d).



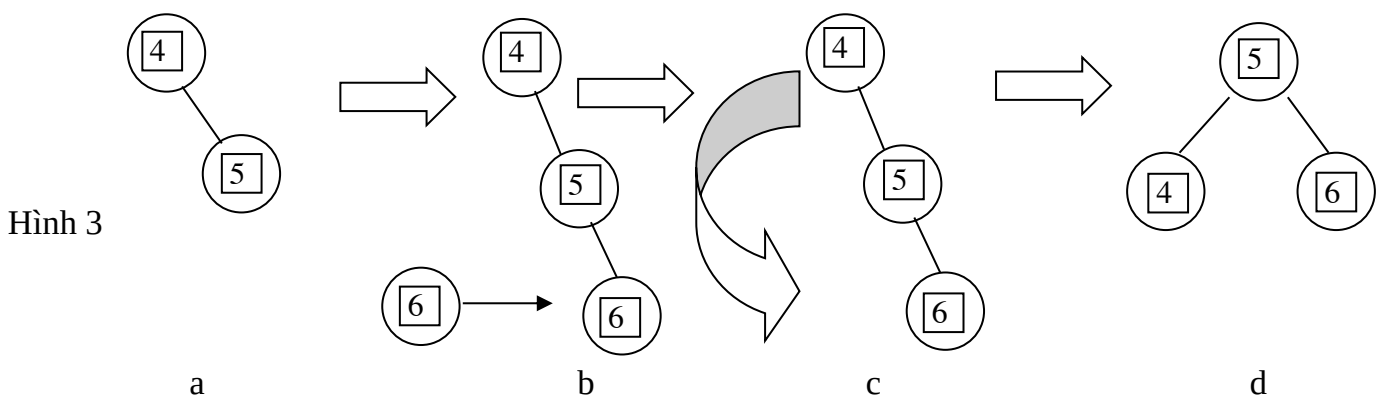
Điều quan trọng phải xét là khi xoay phải thì con trỏ trỏ tới địa chỉ các nút sẽ thay đổi như thế nào. Để cho tiện, ta ký hiệu các nút là x,y,z Ký hiệu $p \rightarrow$ là con trỏ trỏ đến một nút nào đó.



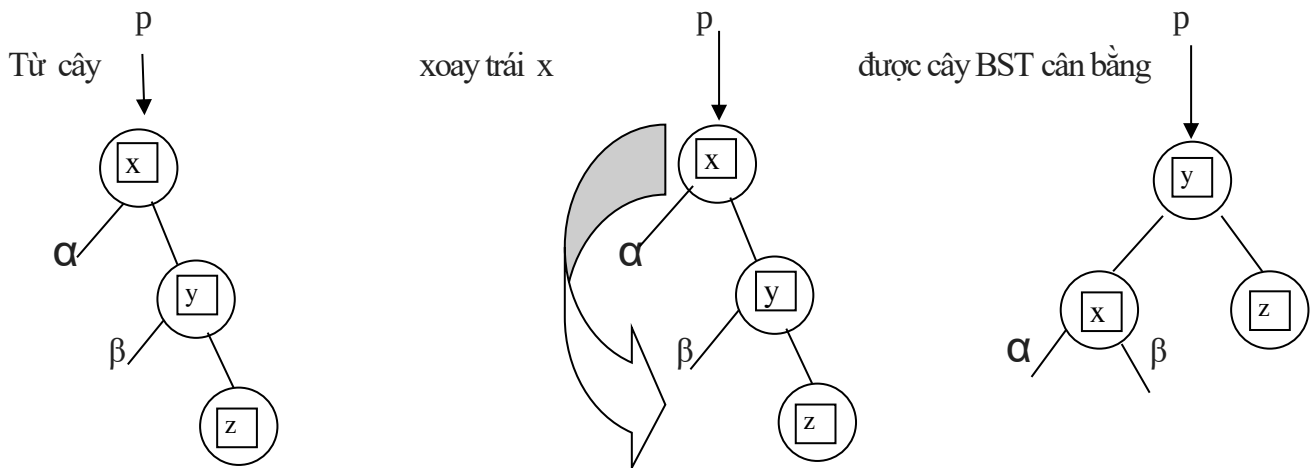
Sau khi thêm **x** vào bên cây con trái của y, cây z mất cân bằng, cần tiến hành xoay.

3.9.2.2. Quay trái cây AVL

Nếu tại một nút của cây trở nên không cân bằng khi một nút mới được chèn vào trong cây con bên phải của cây con bên phải của nút đang xét thì ta quay trái như sau:



Trong hình 3 trên đây : ban đầu ta có cây phải 2 nút 4,5 (h3.a), sau đó thêm nút 6 vào cây con bên phải(h3.b) thành cây không cân bằng. Thực hiện phép quay trái nút 4 (h3.c) thành cây cân bằng (h3.d). Bây giờ xem xét khi xoay trái, cùng với sự thay đổi vị trí các nút thì con trỏ tới địa chỉ các nút sẽ thay đổi như thế nào. Để cho tiện, ta dùng lại ký hiệu các nút là x,y,z như khi quay phải.



Sau khi thêm z vào bên cây con phải của y, cây x mất cân bằng, cần tiến hành xoay.

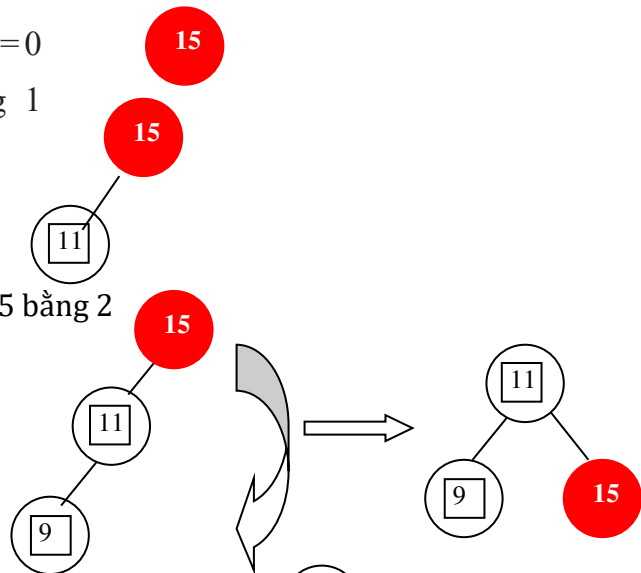
Ví dụ tạo cây với các dữ liệu : 15, 11, 9, 25, 30

Bước 1 : tạo nút gốc 15, kiểm tra hệ số cân bằng = 0

Bước 2 : thêm nút 11, hệ số cân bằng nút 15 bằng 1

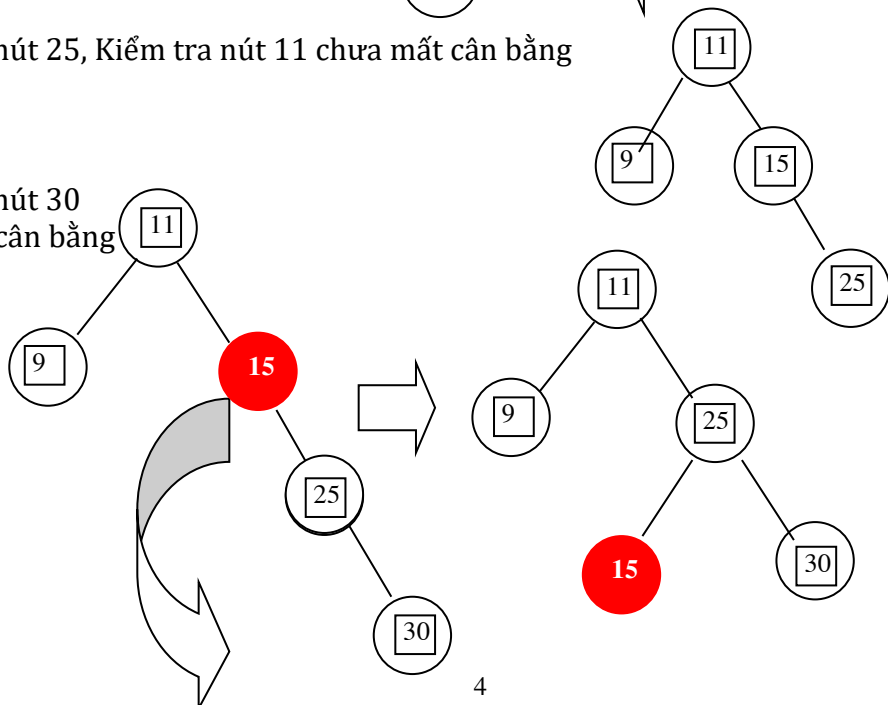
Bước 3 : thêm nút 9, Hệ số cân bằng nút 15 bằng 2

Mất cân bằng, cần **xoay phải**



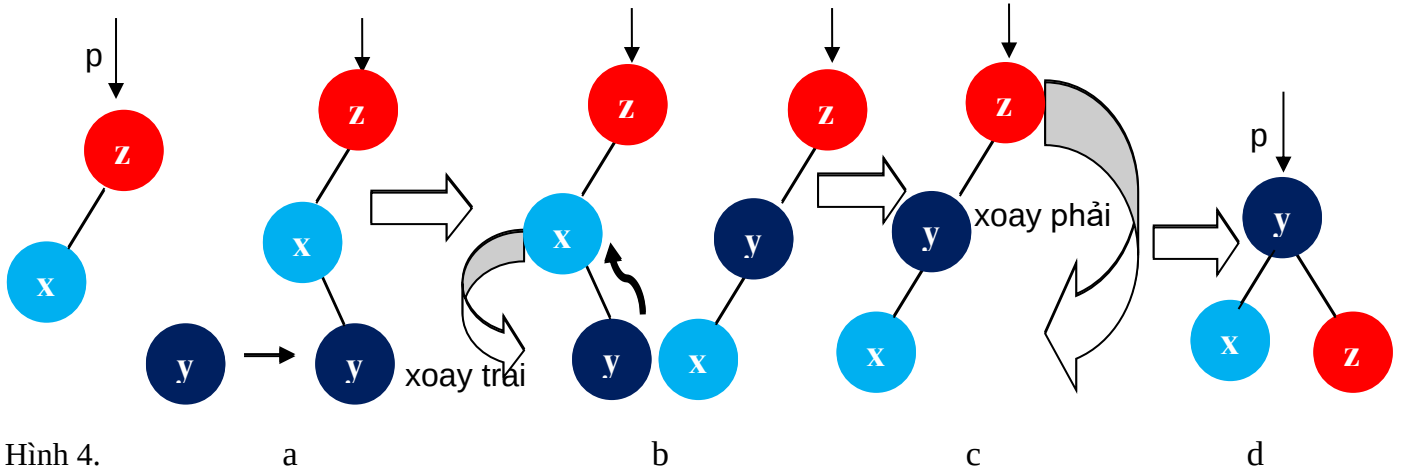
Bước 4: thêm nút 25, Kiểm tra nút 11 chưa mất cân bằng

Bước 5: thêm nút 30
nút 15 sẽ mất cân bằng
cần **xoay trái**



3.9.2.3. Quay trái-phải cây AVL

Phép quay ghép trái-phải là sự kết hợp của quay trái, theo sau là quay phải.

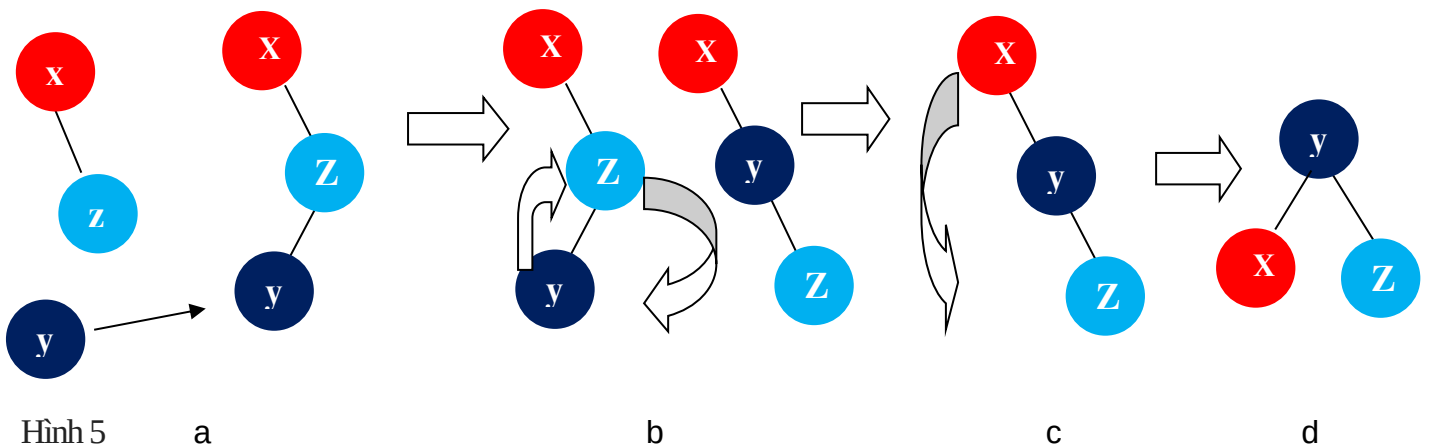


Hình 4.

Trong hình 4.a : lúc đầu có cây trái 2 nút x,z. Sau thêm nút y vào cây con phải của cây con trái nút z, cây không cân bằng. Hình 4.b : thực hiện phép quay trái trên cây con bên trái của nút z, đưa nút x lên trái, nút y thế chỗ nút x. Hình 4.c : thực hiện phép quay phải nút z thành cây con phải của nút y

Hình 4.d : kết quả được cây cân bằng .

3.9.2.4. Quay phải-trái



Hình 5

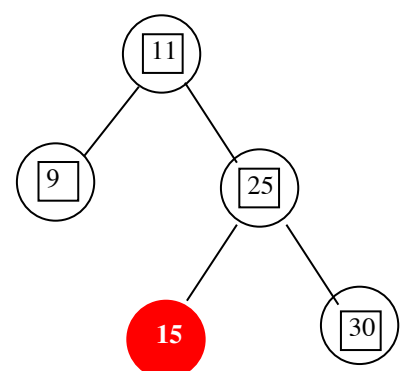
Hình 5.a: nút y được chèn thêm vào cây con trái của cây con phải của nút x, làm mất cân bằng phải-trái

Hình 5.b : quay phải z lên và nút y xuống thay nút z, z thành cây con phải của nút y, x mất cân bằng phải-phải

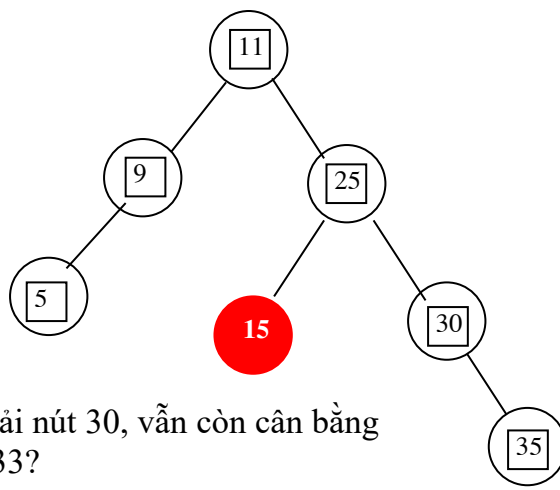
Hình 5.c : quay trái nút x lên thành cây con trái của nút y

Hình 5.d : nút y được cân bằng

Xét tiếp ví dụ trên, ở bước 5 ta đã có cây như hình bên :

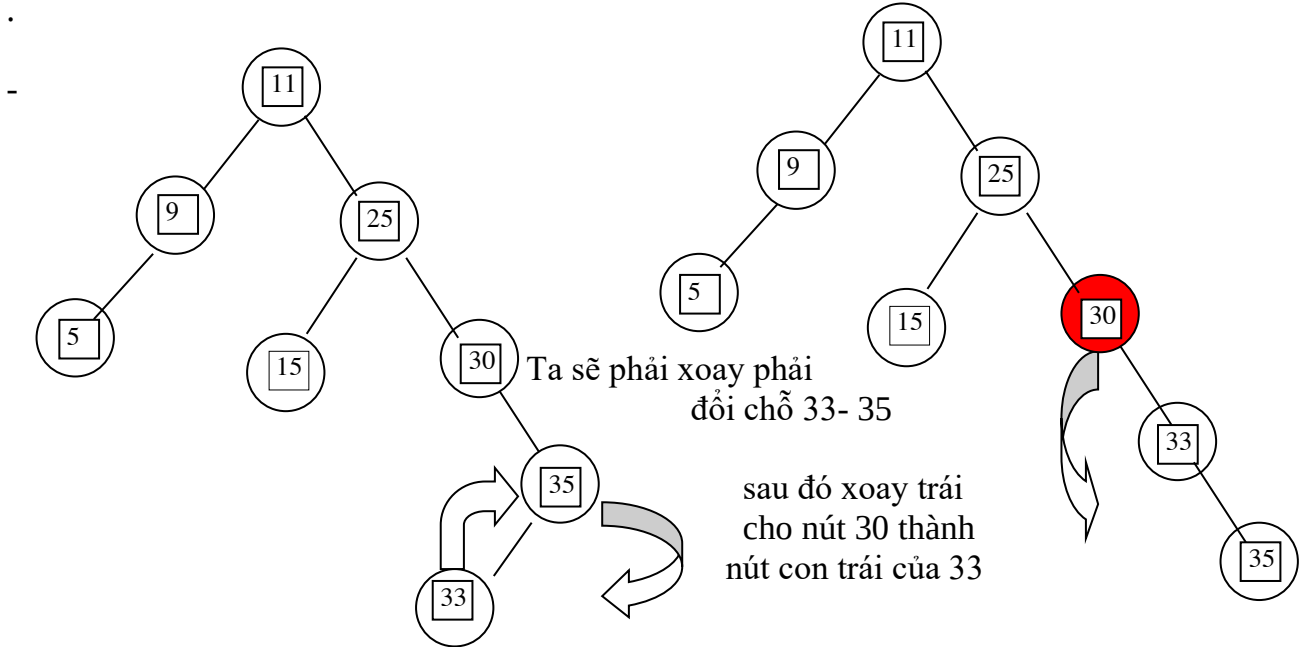


Bây giờ thêm nút có dữ liệu = 5 được đưa vào lá trái nút 9
Chưa mất cân bằng,

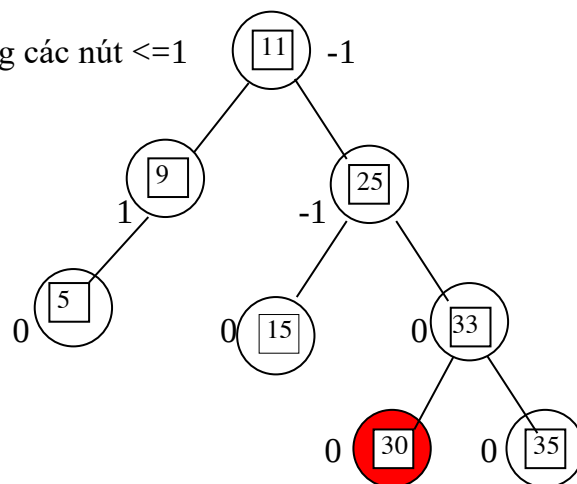


Thêm vào tiếp nút 35 bên phải nút 30, vẫn còn cân bằng
Điều gì xảy ra khi thêm nút 33?

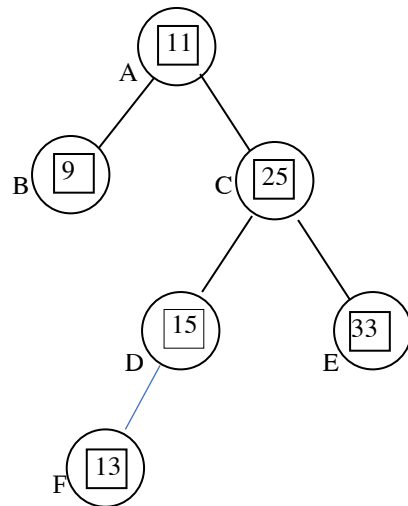
Trước hết là nút 30 mất cân bằng phải – trái,



kết quả hệ số cân bằng các nút ≤ 1



Ví dụ thêm về xoay phải – trái : đã có một cây AVL có 5 nút A-B-C-D-E với dữ liệu nút lần lượt là 11-9-23-15-33 :

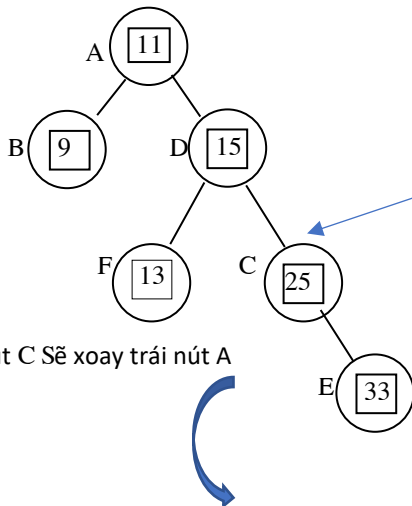


Thêm nút F = 13
vào cây con trái của nút D

Làm cho nút A = 11 mất cân bằng phải – trái, lúc đầu cần xoay phải nút 25



Kết quả :

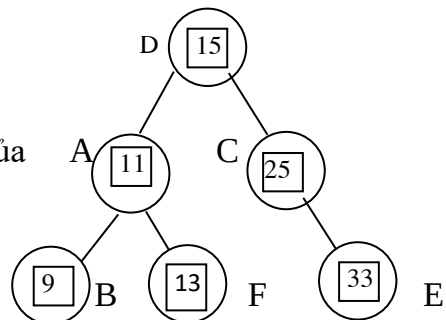


Chú ý : khi xoay phải, nút bị xoay (C=25) sẽ trở thành nút con phải của nút thay thế nó, còn nút con phải của nó vẫn giữ nguyên

Sau khi xoay phải nút C Sẽ xoay trái nút A



Kết quả :



Chú ý : khi xoay nút A, nó trở thành nút con trái của của nút con phải của nó, nút con trái của nút con phải của nó trở thành nút con phải của nó. Việc xoay này làm thay đổi quan hệ :

Trước khi xoay đặt $T1 = D = A \rightarrow \text{Right}$; $(F = T1 \rightarrow \text{Left})$;

$T2 = F = T1 \rightarrow \text{Left}$; sau đó xoay bằng : $A = T1 \rightarrow \text{Left}$; $A \rightarrow \text{Right} = T2$;

Ta rút ra kết luận : khi thêm vào hoặc xóa đi một nút, phải kiểm tra lại hệ số cân bằng của các nút . Nếu hệ số cân bằng của nút bất kỳ khác -1, 0, 1 thì xoay :

Trường hợp A : nếu hệ số cân bằng của nút hiện thời > 1 (mất cân bằng **trái**),

{A1: Và nếu hệ số cân bằng của nút con trái ≥ 0 (trái)

(Tương đương với $du_lieu < node \rightarrow Lchild \rightarrow du_lieu$) thì **xoay phải**

A2 : nếu không ($du_lieu > node \rightarrow left \rightarrow du_lieu$)(phải) thì **xoay trái – phải**}

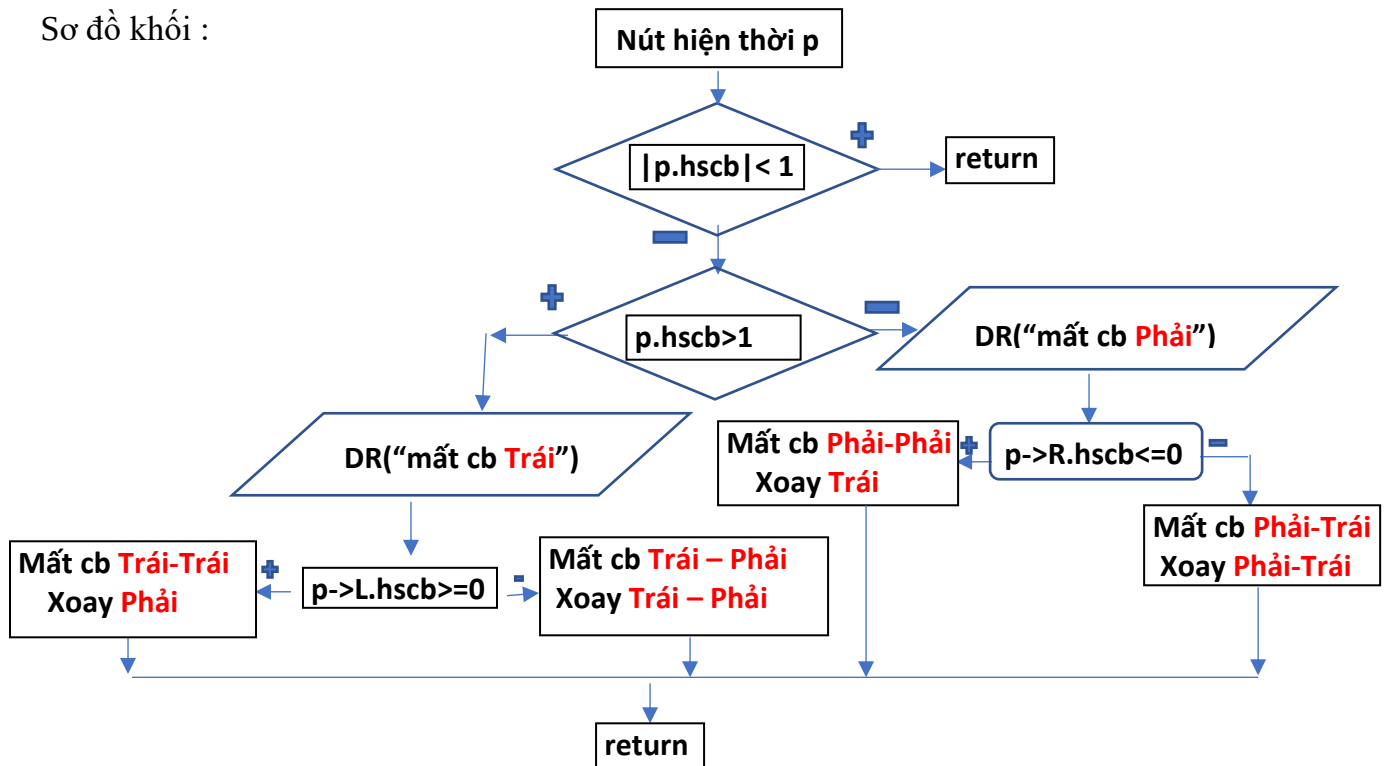
Trường hợp B : nếu hệ số cân bằng của nút hiện thời < -1 (**phải**)

{B1 : Và nếu hệ số cân bằng của nút con trái ≤ 0 (**phải**)

(tương đương $du_lieu > node \rightarrow Rchild \rightarrow du_lieu$) thì **xoay trái**

B2: nếu không ($du_lieu < node \rightarrow Rchild \rightarrow du_lieu$)(**trái**) thì xoay **phải- trái**}

Sơ đồ khối :



Để dễ nhớ, cứ hệ số cân bằng của nút đang xét > 1 là mất cân bằng TRÁI (cạnh và nút ở Trái) đến nút con của nó, nếu hệ số cân bằng ≥ 0 là mất cân bằng trái (cạnh và nút ở Trái), ta có dạng Trái-Trái, dẫn tới xoay Phải.

Ngược lại, nút con của nó có hệ số cân bằng ≤ 0 là mất cân bằng phải, ta có dạng Trái – Phải, dẫn tới xoay Trái – Phải.

Dạng thứ 2 là hệ số cân bằng của nút đang xét < -1 là mất cân bằng PHẢI (cạnh và nút ở bên Phải), đến nút con của nó, nếu hệ số cân bằng ≤ 0 là mất cân bằng Phải (cạnh và nút ở Phải), ta có dạng Phải-Phải, dẫn tới xoay Trái.

Ngược lại, nút con của nó có hệ số cân bằng ≥ 0 là mất cân bằng trái (cạnh và nút ở trái), ta có dạng Phải - Trái, dẫn tới xoay Phải - Trái.

3.9.3. Cài đặt cây AVL

1. Khai báo cấu trúc cây AVL :

```

struct AvlNode
{
    int du_lieu;
    AvlNode *Lchild;
    AvlNode *Rchild;
    int height;
};
    
```

2. Tạo nút mới :

```

AvlNode *TaoNode(int du_lieu) {
    AvlNode *node = new AvlNode;
    node->du_lieu = du_lieu;
    node->Lchild = NULL;
    node->Rchild = NULL;
    node->height = 1;
    return node;
}
    
```


}

3. Xác định hệ số cân bằng - hscb:

```
int hscb(AvlNode *GOC)
{ if (GOC == NULL)
  return 0;
  return height(GOC->Lchild) - height(GOC->Rchild);
}
```

4. Xoay Phải

Ta xem xét trường nhập nút tạo cây, kiểm tra nếu mất cân bằng trái thì xoay phải.

Bước 1 : nhập nút gốc y=50, cây đang cân bằng;

Bước 2: Thêm nút x=45 vào bên trái nút 50; cây đang cân bằng;

Bước 3: Thêm nút z=40 vào bên trái nút 45;
cây mất cân bằng dạng trái-trái tại nút gốc 50;

Cần Xoay Phải :

Nút hiện thời mất cân bằng là y, có dữ liệu = 50

(giả thiết y đang ở địa chỉ **0xe317c0**; Con trỏ y đang ở địa chỉ : 0x61fda0).

Nút x = y->left có dữ liệu = 45 (giả thiết đang ở địa chỉ **0xe31830**)

Ta tạo 1 biến phụ p = x->right;

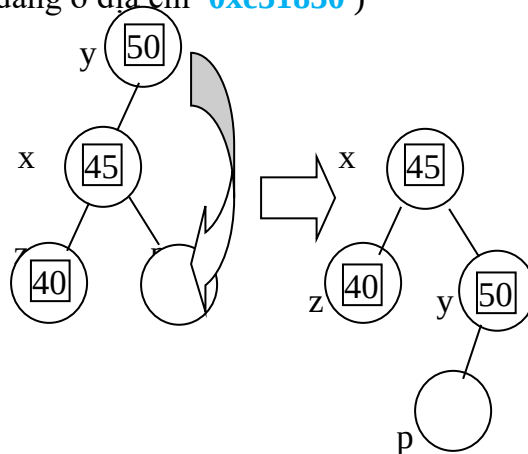
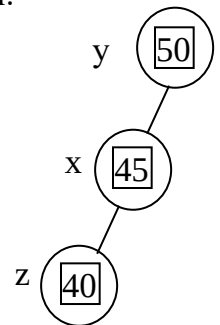
Xoay phải => : x->right = y (0xe317c0)

y->left = p; ; 0

y->height = 1

x->height = 2

Trả về x tại địa chỉ **0xe31830** .



Ta có hàm :

```
AvlNode *XoayPhai(AvlNode *y) {
  AvlNode *x = y->Lchild;
  AvlNode *p = x->Rchild;
  x->Rchild = y; // xoay phải y lên thành nút con phải của x
  y->Lchild = p;
  y->height = max(height(y->Lchild), height(y->Rchild)) + 1;
  x->height = max(height(x->Lchild), height(x->Rchild)) + 1;
  return x;
}
```

5. Xoay Trái :

Ta xét tiếp ví dụ trên sau khi xoay phải về thể cân bằng, nhập nút tạo cây, kiểm tra nếu mất cân bằng trái thì xoay trái.

Bước 4 : nhập thêm nút 55, cây đang cân bằng;



Bước 5: Thêm nút 60 vào bên phải nút 55

Làm mất cân bằng dạng phải-phải tại nút 50

Cần Xoay Trái :

Nút hiện thời x = **50** (giả sử tại địa chỉ **0x1b17c0**)

y = x->right có dữ liệu = **55** (giả sử ở địa chỉ **0x1b1890**)

Tạo biến phụ p = y->left ;// *p đang trống, không có dữ liệu*

Xoay trái => y->left = x : đưa x=50 vào y->left tại địa chỉ **0x1b17c0**

Đưa p lên làm nút con phải của x : x->right = p

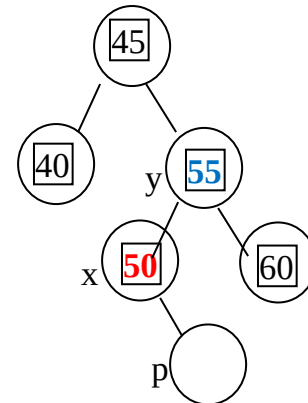
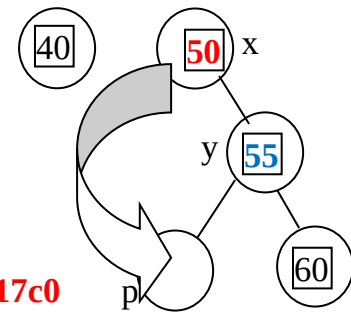
. x->height = 1

y->height = 2

Trả về y= **55** tại địa chỉ **0x1b1890**

Ta có hàm :

```
AvlNode *XoayTrai(AvlNode *x) {
    AvlNode *y = x->Rchild;
    AvlNode *p = y->Lchild;
    y->Lchild = x;
    x->Rchild = p;
    x->height = max(height(x->Lchild), height(x->Rchild)) + 1;
    y->height = max(height(y->Lchild), height(y->Rchild)) + 1;
    return y;
}
```



6. Thêm nút mới

Một nút mới được thêm vào bao giờ cũng là một nút lá và có hệ số cân bằng = 0.

```
AvlNode *ChenNode(AvlNode *node, int du_lieu) {
    // Tìm vị trí thích hợp để thêm nút
    if (node == NULL)
        return (TaoNode(du_lieu));
    if (du_lieu < node->du_lieu)
        node->Lchild = ChenNode(node->Lchild, du_lieu);
    else if (du_lieu > node->du_lieu)
        node->Rchild = ChenNode(node->Rchild, du_lieu);
    else
        return node;
    // Kiểm tra hệ số cân bằng và cân bằng cây
    node->height = 1 + max(height(node->Lchild),
        height(node->Rchild));
    int hs = hscb(node);
    if (hs > 1 && du_lieu < node->Lchild->du_lieu)
        return XoayPhai(node);
    if (hs < -1 && du_lieu > node->Rchild->du_lieu)
        return XoayTrai(node);
    if (hs > 1 && du_lieu > node->left->du_lieu) {
        node->left = XoayTrai(node->Lchild);
    }
}
```

```

    return XoayPhai(node);
}
if (hs < -1 && du_lieu < node->Rchild->du_lieu) {
    node->Rchild = XoayPhai(node->Rchild);
    return XoayTrai(node);
}
return node;
}

```

Bài tập.

1. Tập làm thêm các ví dụ khác về các phép xoay cây AVL
2. Vẽ sơ đồ thuật toán về cách xoay trái/phải
3. Dựa trên phần cài đặt cây AVL đã cho, viết chương trình C++ tạo cây AVL với các nút 11-9-23-15-33 lần lượt được nhập vào bằng hàm `ChenNode(AvlNode *node, int du_lieu)` ;

3.10.CÂY HEAP

3.10.1. Khái niệm Heap (Vun đống / khối)

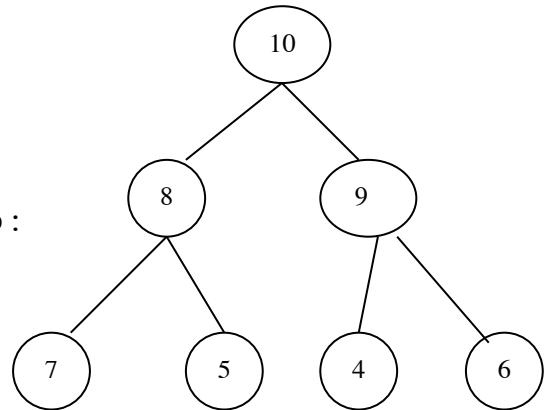
Vun đống là một loại cây nhị phân đặc biệt có các đặc điểm:

- Nó là một cây đầy đủ (cây hoàn chỉnh), nghĩa là các lá trên cây nằm nhiều nhất ở hai mức liên tiếp nhau và các lá ở mức dưới nằm ở những vị trí bên trái nhất (leftmost positions)
- Giá trị trong mỗi nút cha lớn hơn các giá trị ở các nút con gọi là Max Heap;
- Giá trị ở các nút con lớn hơn giá trị nút cha gọi là Min Heap;

Các thao tác thường dùng với Heap là

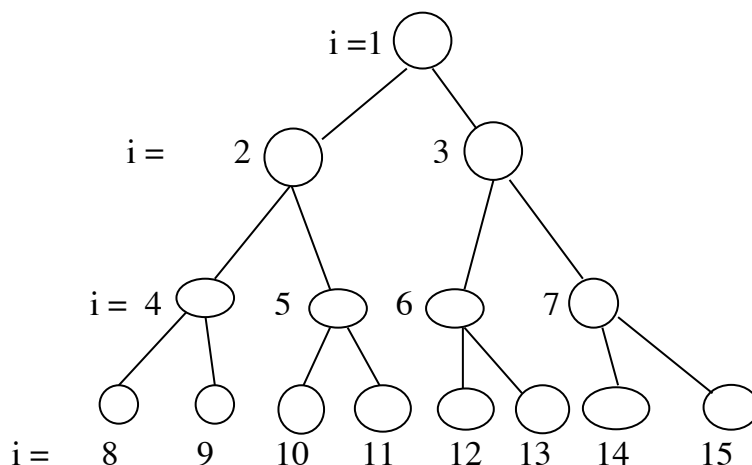
- ✓ Cài đặt Max Heap/Min Heap, Xây dựng Heap từ một tập N phần tử
- ✓ Tìm nút lớn nhất/nhỏ nhất trên Heap
- ✓ Thêm một nút vào Heap
- ✓ Xóa bỏ nút lớn nhất/nhỏ nhất trên Heap

Ví dụ Max Heap :



3.10.2.Max Heap

Để cài đặt Heap ta có thể dùng cấu trúc liên kết nh- đối với BST, song có thể dùng mảng, đánh số thứ tự từ trên xuống, ở mỗi mức các nút đ- ọc đánh số từ trái qua phải:



i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
H(i)															

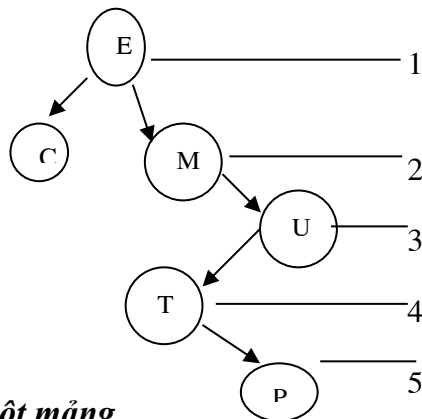
Tính đầy đủ của Heap bảo đảm cho các mục dữ liệu đ- ọc l- u trữ trong các vị trí liên tiếp ở đầu mảng. Có thể khai báo mảng Heap nh- sau:

```

const Sonut = ... { số nút cực đại trong khối }
typedef Heaptyp kieuphant- = ... { kieudữ liệu}
Heaptyp Heap;
  
```

Trong cách cài đặt mảng nh- vậy việc tìm con của nút thứ i sẽ ở vị trí $2*i$ và $2*i+1$, bố của nút thứ i nằm ở vị trí $(i/2)$.

Đối với các loại cây nhị phân khác, cài đặt mảng là không hiệu quả:



.Xây dựng Max Heap từ một mảng

Giả sử ta có một cây nhị phân hoàn thiện, dữ liệu các nút của nó được lưu trong mảng dữ liệu chưa được sắp trật tự , cần chuyển đổi nó thành một mảng Max Heap, lưu trữ các node của heap như cây Max Heap nhị phân ở trên.

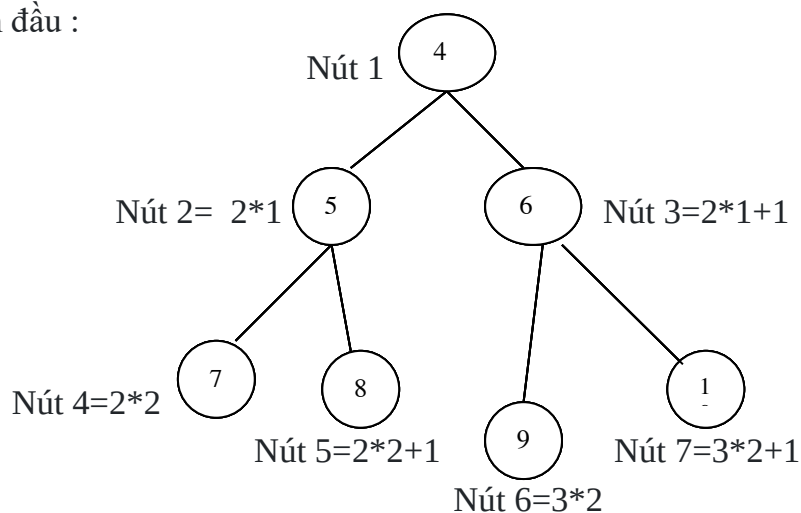
Các thao tác cần làm là :

- Lấy một phần tử từ mảng tạo Heap.
- Kiểm tra xem con bên trái của nó có phải max heap không.
- Kiểm tra xem con bên phải của nó có phải max heap không.
- Kiểm tra xem chính node đó có phải max heap không.

Chú ý tính chất của Max Heap : Giá trị trong mỗi nút cha lớn hơn các giá trị ở các nút con

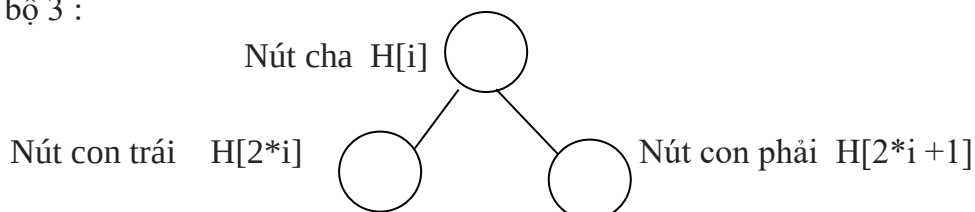
Trong cách cài đặt mảng, việc tìm con của nút thứ i sẽ ở vị trí $2*i$ và $2*i+1$, bố của nút thứ i nằm ở vị trí $(i/2)$.

Xét cây nhị phân ban đầu :



i	1	2	3	4	5	6	7
$H[i]$	4	5	6	7	8	9	10

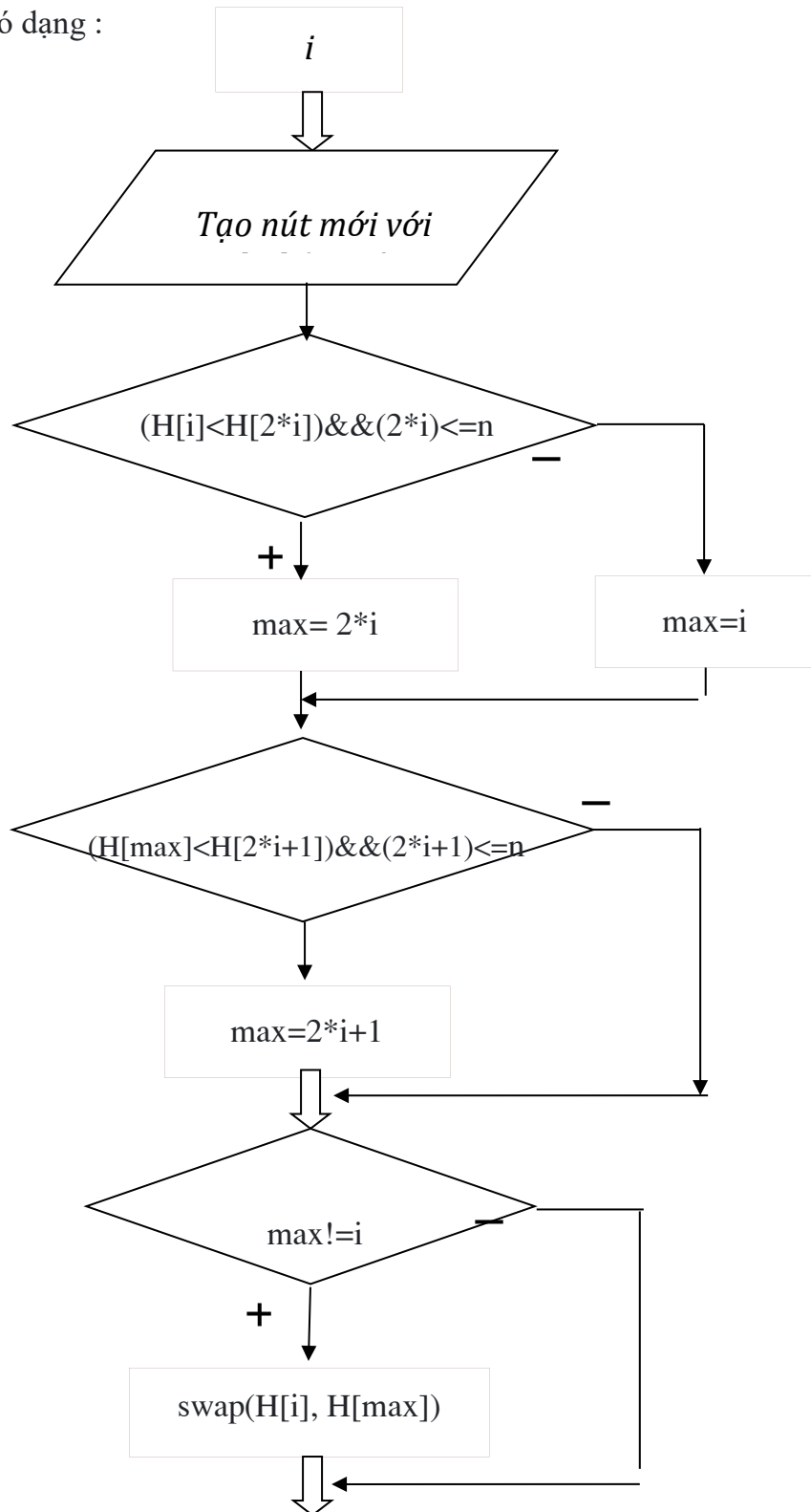
Để tạo dựng một cây Max-Heap, tại bất kỳ một thời điểm thứ i nào ta đưa thêm một nút thì phải xét nó trong bộ 3 :



Để đảm bảo thuộc tính MaxHeap là giá trị dữ liệu trong nút cha thứ i lớn hơn giá trị dữ liệu trong 2 nút con thứ $2*i$ và $2*i+1$, ta phải xác định giá trị nút mới thêm, nếu cần thì hoán đổi vị trí sao cho sau khi thêm có được

$$H[i] = \text{Max}\{H[i], H[2*i], H[2*i+1]\}$$

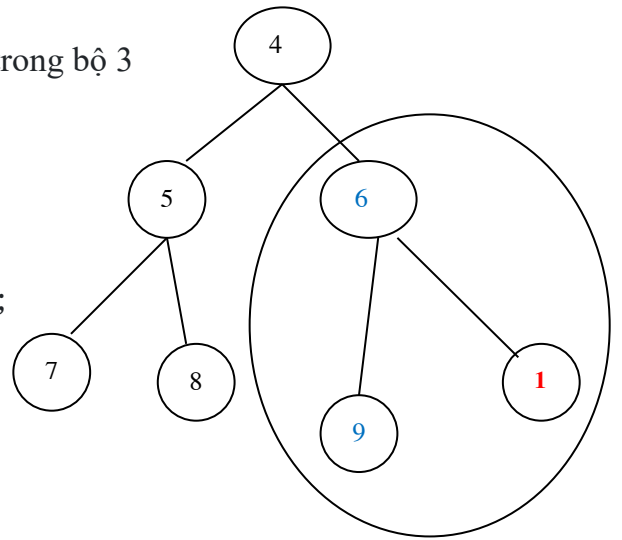
Sơ đồ thuật toán phần này có dạng :



Hàm xét maxHeap

void maxHeap(int H[], int i) // sử dụng n là số phần tử của mảng H[], biến toàn cục

```
{ int max; // max là chỉ số của nút có giá trị lớn nhất trong bộ 3
  int Lchild=2*i, Rchild=2*i + 1;
  if(Lchild<= n && H[Lchild]>H[i]) max= Lchild;
  else
    max = i ;
  if(Rchild<= n && H[Rchild]>H[max]) max= Rchild;
  if(max != i)
    { swap(H[i], H[max]);
      maxHeap(H, max);
    }
}
```



Độ phức tạp về thời gian: $O(\log N)$

Chú ý : hàm maxHeap không cần áp dụng với các node lá, vì các node lá không có nút con. Do vậy ta chỉ cần áp dụng hàm maxheap() cho các phần tử còn lại được đánh chỉ số từ 1 tới $n/2$.

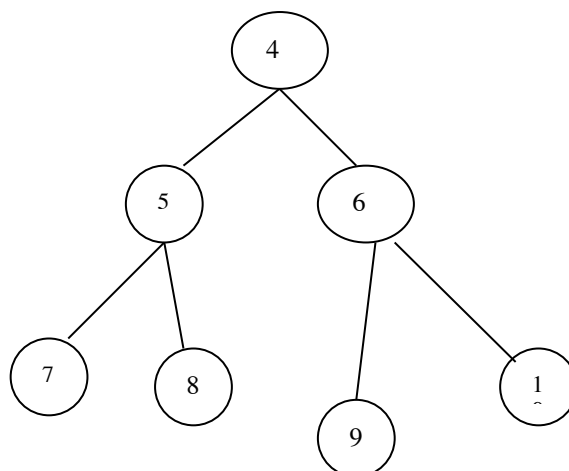
Vì mỗi node lá là một thành phần của max Heap nên ta áp dụng hàm maxHeap theo hướng từ dưới lên, để ta có thể bao quát được toàn bộ cây.

Hàm Tạo MaxHeap

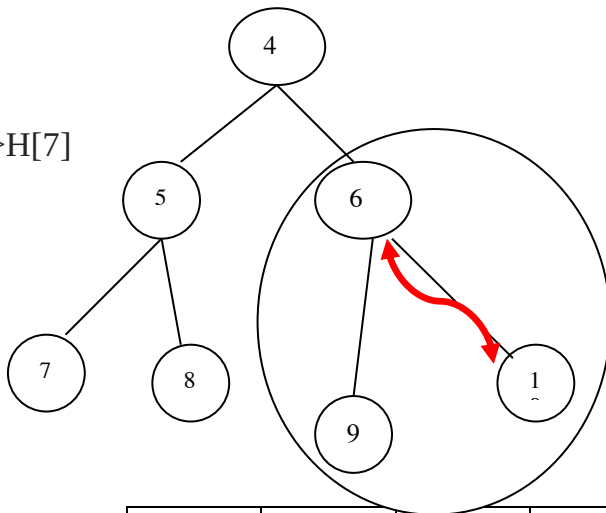
```
void TaomaxHeap(int H[])
{for(int i=n/2;i>=1;i--)
  { maxHeap(H,i);  }
}
```

Ta sẽ xem ví dụ với mảng H[] ban đầu , $n=7$: hàm maxHeap từ vị trí $n/2 = 3$ tới 1.

i	1	2	3	4	5	6	7
H[i]	4	5	6	7	8	9	10



Với $i=3$
 $\text{max}=7$
 $H[3] \leftrightarrow H[7]$

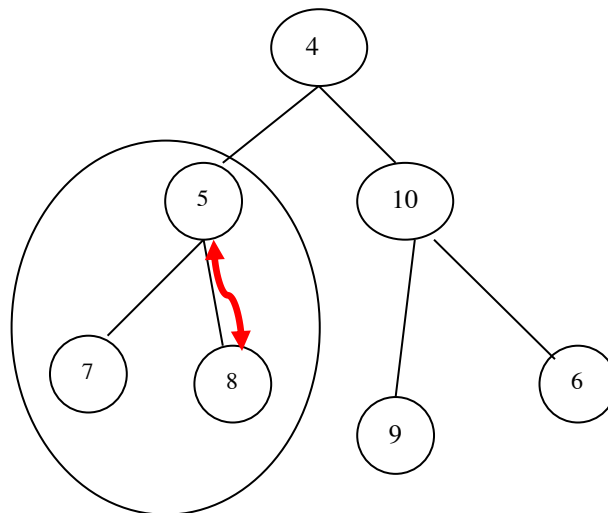


Kết quả :

i	1	2	3	4	5	6	7
H[i]	4	5	10	5	8	9	6

Khi $i = 2$

$\text{max}=5$
 $H[2] \leftrightarrow H[5]$

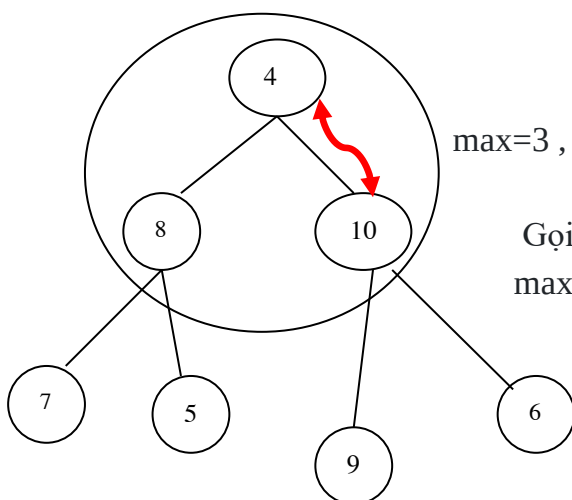


Kết
quả :

i	1	2	3	4	5	6	7
H[i]	4	8	10	7	5	9	6

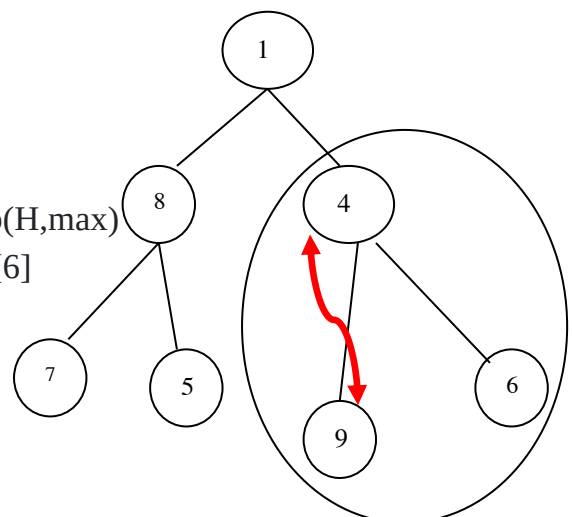
Khi $i = 1$

$\text{Lchild}=2, \text{Rchild}=3$



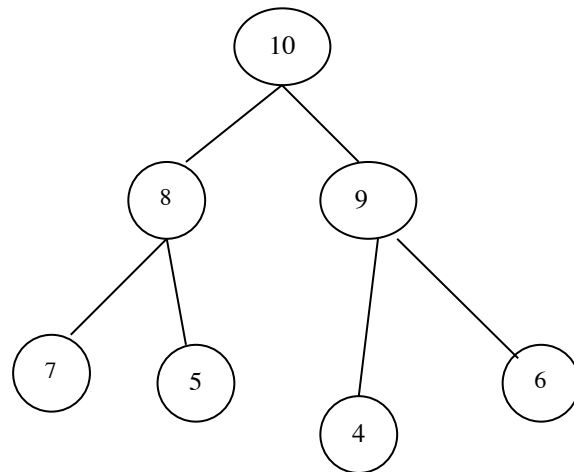
$\text{max}=3, H[1] \leftrightarrow H[3];$

Gọi đệ quy $\text{maxHeap}(H, \text{max})$
 $\text{max}=6$, sẽ $H[3] \leftrightarrow H[6]$



Kết
quả :

i	1	2	3	4	5	6	7
H[i]	10	8	9	7	5	4	6



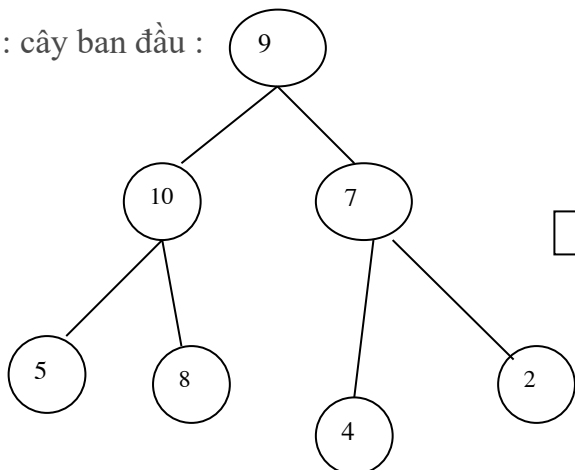
3.10.3. Min Heap

1. Khái niệm

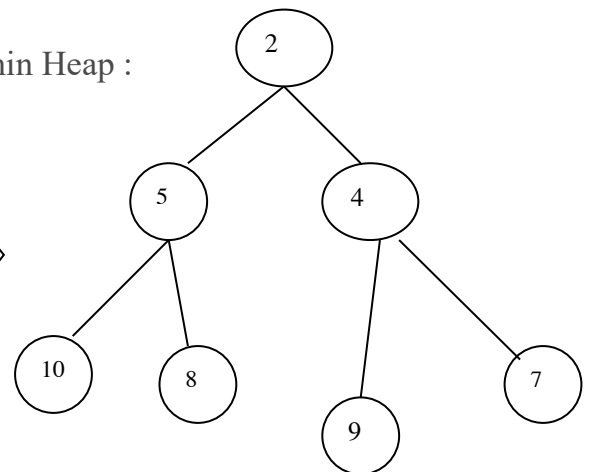
Trong cây Min Heap, giá trị của node cha luôn nhỏ hơn hoặc bằng giá trị của 2 node con. Node gốc của min heap có giá trị nhỏ nhất trong cây.

2. Đổi chỗ dữ liệu các nút trong cây Min Heap

Ví dụ : cây ban đầu :



Cây min Heap :



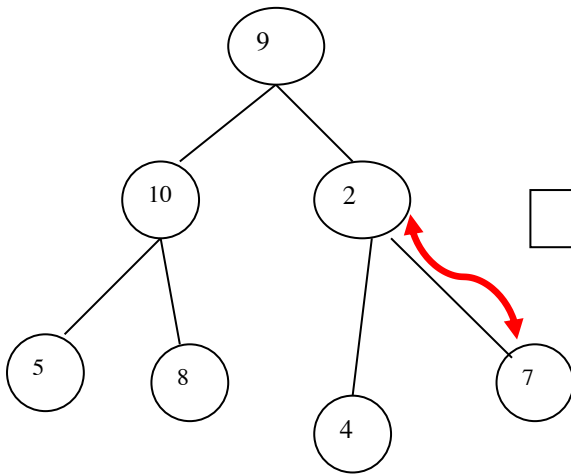
Mảng H ban đầu có n=7 nút:

i	1	2	3	4	5	6	7
H[i]	9	10	7	5	8	4	2

Trong bộ 3 nút cha thứ i với 2 nút con thứ $2*i$ và $2*i+1$ luôn phải đảm bảo dữ liệu nút cha nhỏ hơn dữ liệu của 2 nút con. Quá trình chuyển đổi dữ liệu trong cây để trở thành cây min Heap hàm này chỉ áp dụng cho các nút $i = n/2$ về đến nút 1.

Khi $i = n/2 = 3$. Xét bộ 3 nút thứ 3, thứ 6,7. Min ở nút thứ 7,

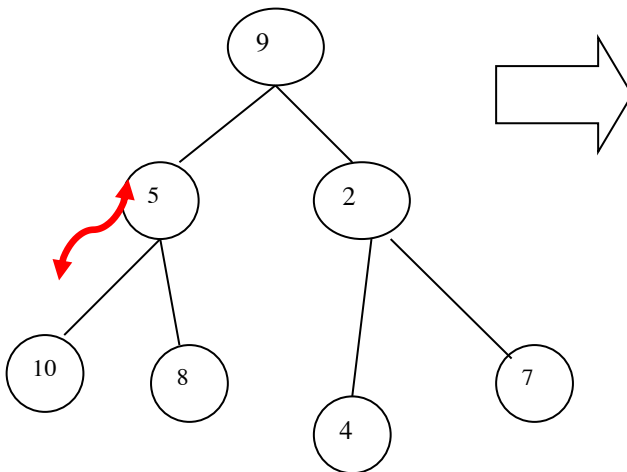
đổi chỗ $H[3] \leftrightarrow H[7]$



Khi $i = 3$:

$H[1] = 9$
$H[2] = 10$
$H[3] = 2$
$H[4] = 5$
$H[5] = 8$
$H[6] = 4$
$H[7] = 7$

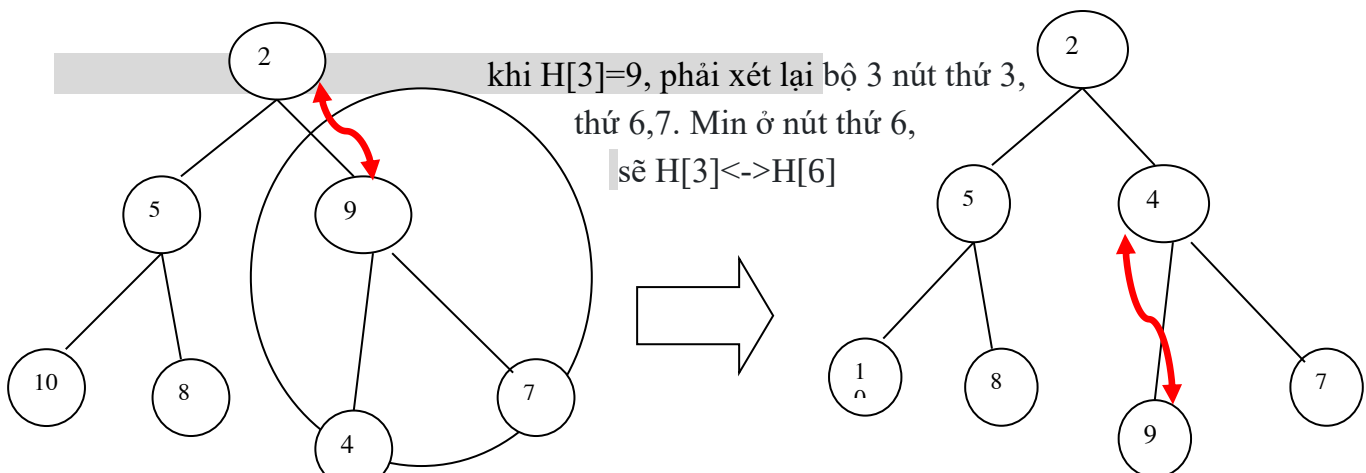
Khi $i = 2$, $H[2] = 5$, Xét bộ 3 nút thứ 2, thứ 4, 5. Min ở nút thứ 4, đổi chỗ $H[2] \leftrightarrow H[4]$



Khi $i = 2$:

$H[1] = 9$
$H[2] = 5$
$H[3] = 2$
$H[4] = 10$
$H[5] = 8$
$H[6] = 4$
$H[7] = 7$

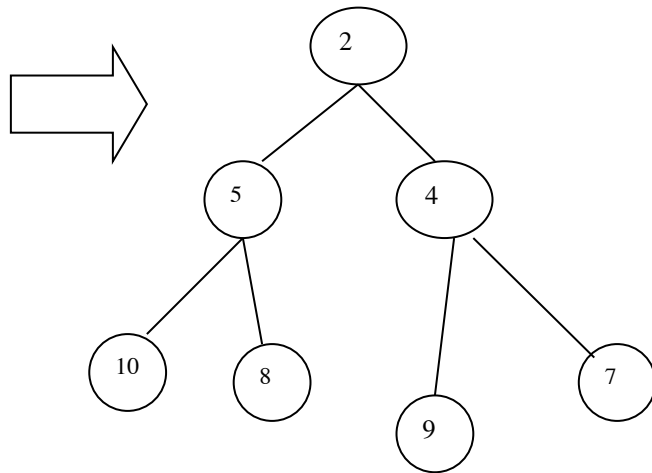
Khi $i = 1$, $H[1] = 8$, Xét bộ 3 nút thứ 1, thứ 2, 3. Min ở nút thứ 3, đổi chỗ $H[1] \leftrightarrow H[3]$



```

Khi i = 1:
H[1] = 2
H[2] = 5
H[3] = 4
H[4] = 10
H[5] = 8
H[6] = 9
H[7] = 7

```



Ta có hàm minHeap :

```

void minHeap(int H[], int i) {
    int min; // Chỉ số của phần tử nhỏ nhất trong bộ ba : nút i , 2*i và 2*i+1
    int Lchild = 2 * i; // Vị trí của con bên trái
    int Rchild = 2 * i + 1; // Vị trí của con bên phải
    if (Lchild <= n and H[Lchild] < H[i])
        min = Lchild;
    else
        min = i;
    if (Rchild <= n and H[Rchild] < H[min])
        min = Rchild;
    if (min != i) {
        swap(H[i], H[min]); //đổi chỗ nếu nút cha > nút con
        minHeap(H, min); // Gọi đệ quy node tại vị trí mới
    }
}

```

3. Tạo cây Min Heap

Hàm minHeap() trên đây không tác động các nút lá vì chúng không có nút con, hàm này chỉ áp dụng cho các nút $i = n/2$ về đến nút 1, n là số nút trong cây, là số lượng phần tử trong mảng $H[]$.

Ta có hàm tạo min heap :

```

Void TaoMinHeap(int H[])
{ for(int i=n/2;i>=1;i--)
    {minHeap(H,i);
    cout<<" Khi i = "<<i<<" : " ;

```

```

for (int j = 1; j < n+1; j++)
    { cout<<"\n "<<" H["<<j<<"]= "<<H[j];  }
    cout<<"\n\n";
    system("pause");}
}

```

Bài tập :1) Dùng bút và giấy tạo cây và tập 4 dạng xoay cây AVL

2) Dùng bút và giấy tạo cây Max Heap, tìm hiểu hàm maxheap và TaomaxHeap(int H[])

3) Viết chương trình maxheap và MinHeap, chạy lại với các bộ dữ liệu mới.